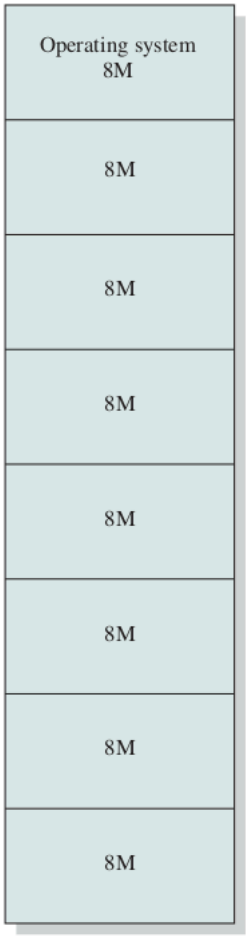
Operating Systems

Memory Management

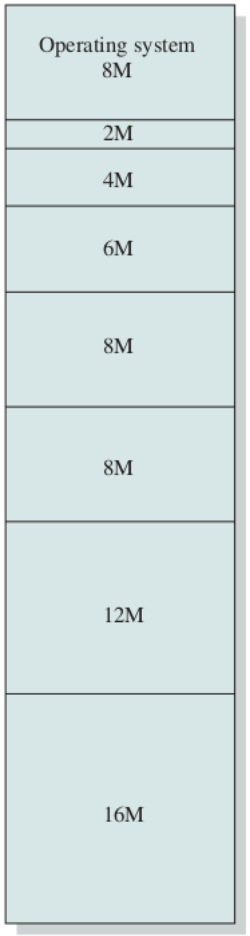
Ahmad Yoosofan

University of Kashan

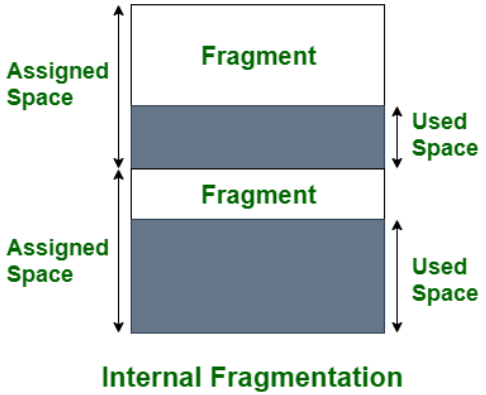
Fixed Partitioining or Static Partitioning



(a) Equal-size partitions

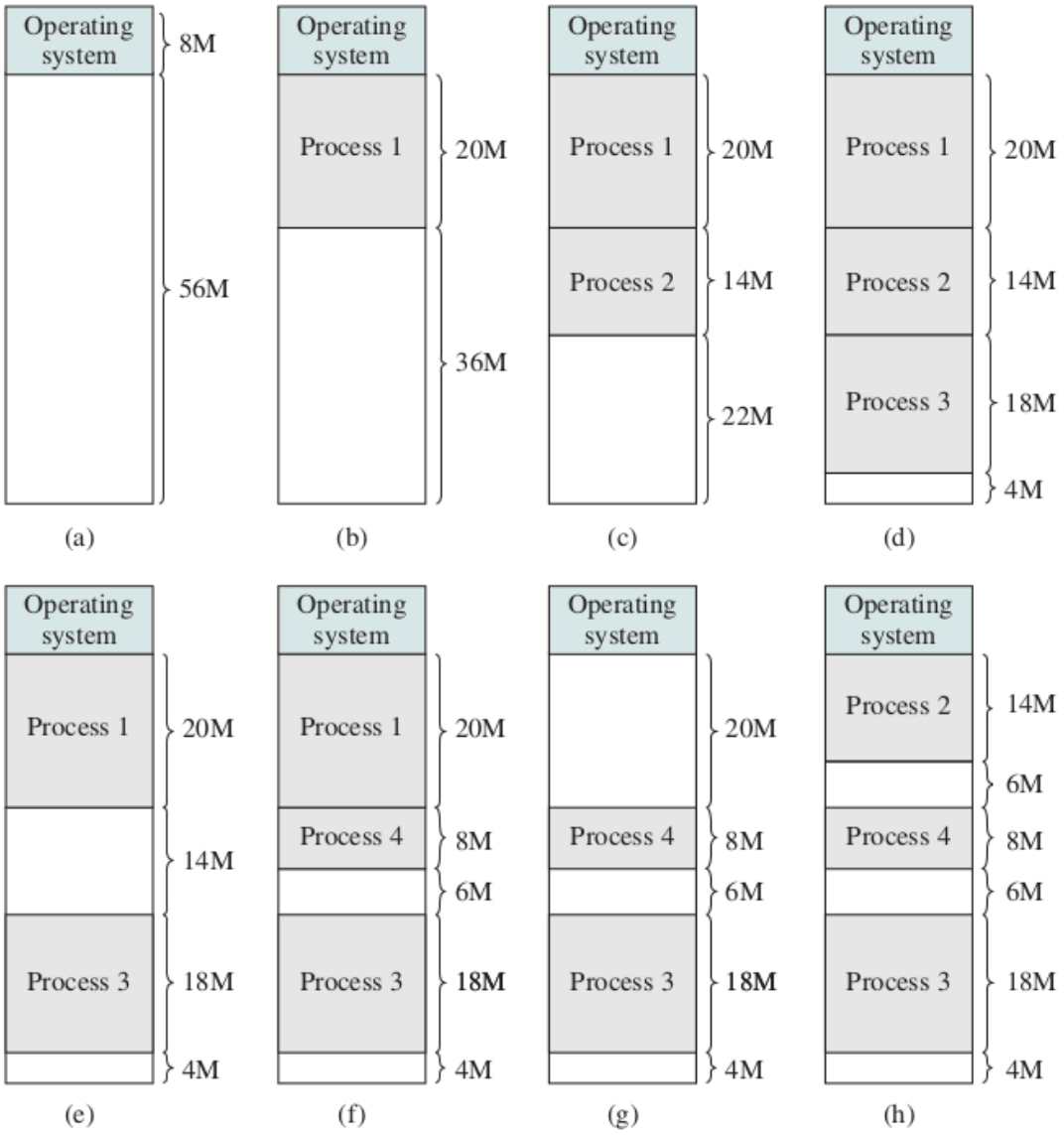


(b) Unequal-size partitions

- Equal-Size Partitions
- Unequal-Size Partitions
- Pros:
    - Minimal Hardware Complexity
    - Simple OS Logic
    - Low Overhead
- Cons:
    - Internal Fragmentation
    - Fixed Process Limit
    - Maximum Size Limit
- 

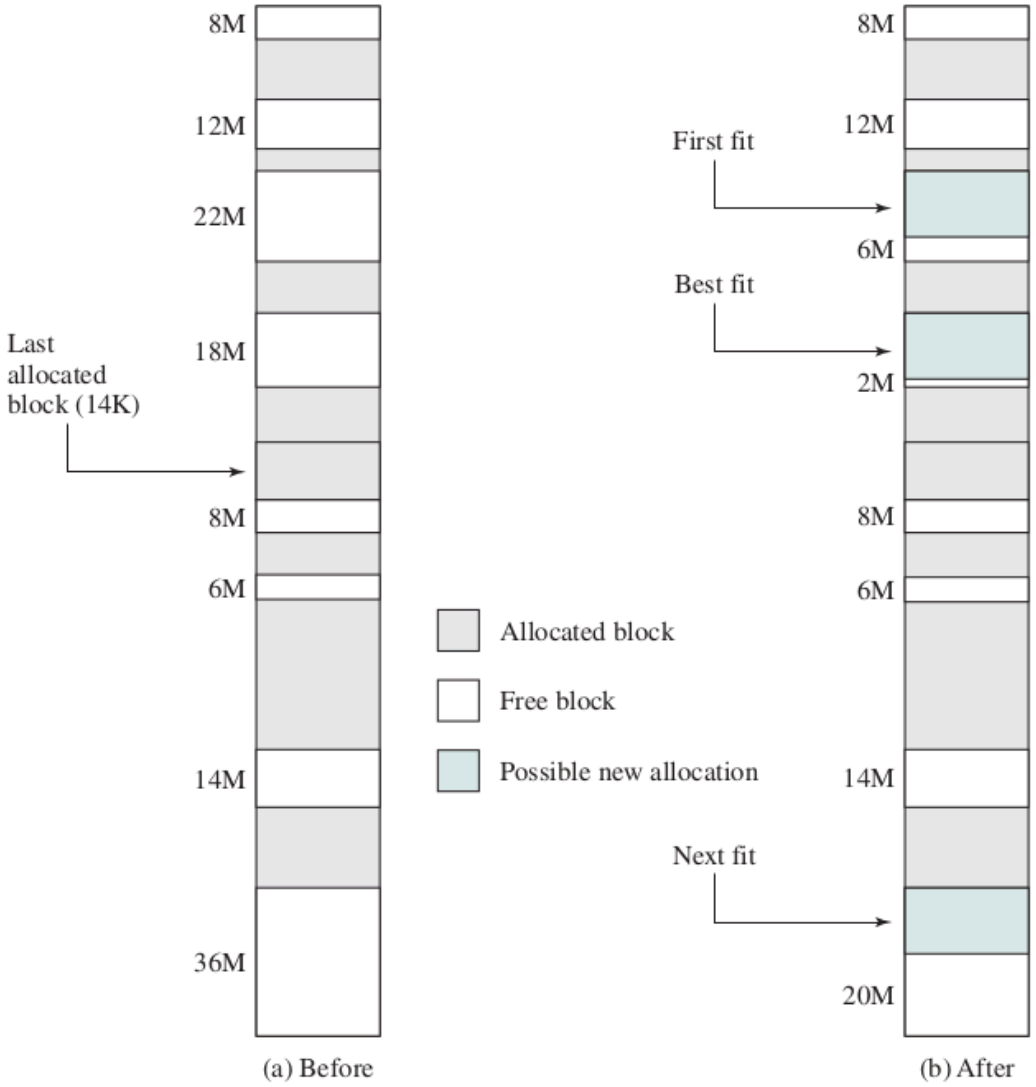
Internal Fragmentation

Dynamic Partitioning



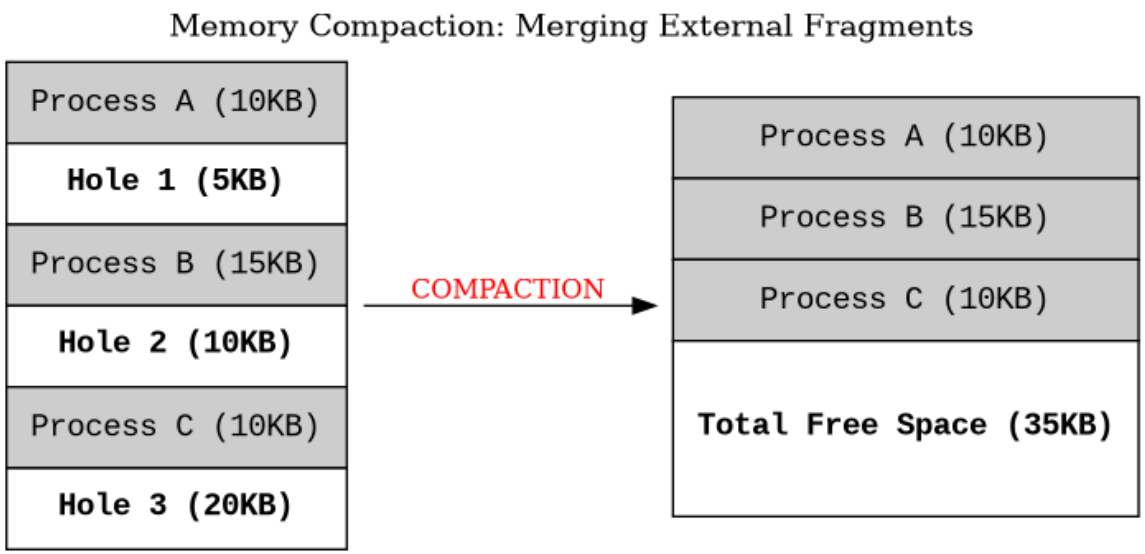
- Hole Tracking or free space tracking
  - Bitmaps
  - Linked Lists
- External fragmentation
Feature	Fixed	Dynamic
Partition Size	Static	Dynamic
Fragmentation	Internal	External
Efficiency	Low	High
- Coalescing (The "Neighborly" Cleanup)
  - Merge free space
- compaction

Placement algorithms



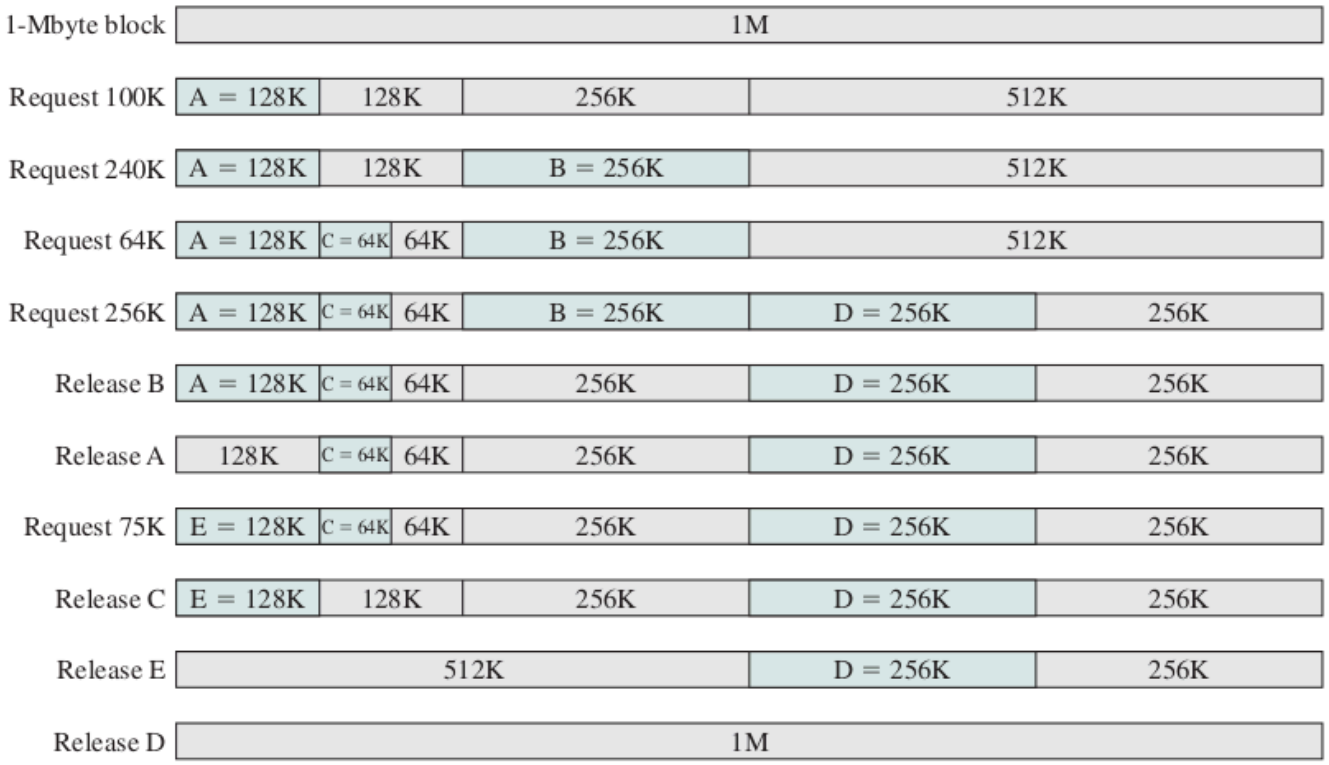
- First Fit
  - pros: fast
  - cons: clog
- Best Fit
  - pros:
  - cons: Slow, Small holes
- Worst Fit
  - pros: large leftover
  - cons: slow, use largest
- Next Fit
  - pros: Prevents clogging
  - cons:
- Quick Fit or Segregated Fit (different lists)
  - pros: instant allocation
  - cons: overhead of OS

Compaction or Defragmentation



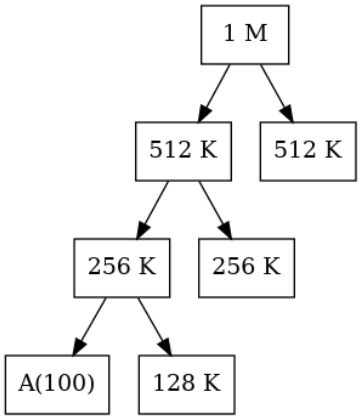
- cost
  - CPU Cycles
  - Bus Saturation
  - System Freeze
- Strategies
  - Move to One End
  - Minimal Movement
- Trigger
  - Allocation Failure
  - Idle Time
  - Threshold
- Pros
  - Maximizes RAM Utilization
  - Enables Large Processes
  - Simplifies Placement
- Cons
  - Extreme Overhead
  - Requires Special Hardware
  - Interrupts Execution

Buddy System Memory Management(I)

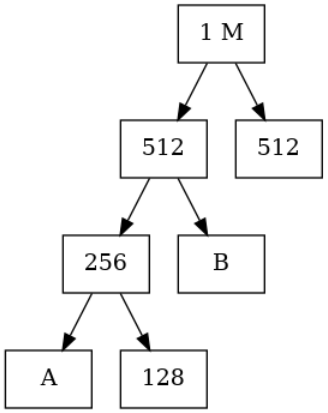


Buddy System Memory Management(II)

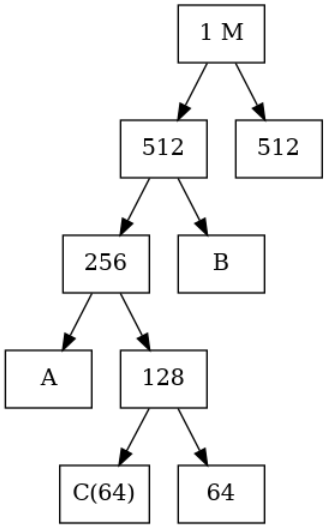
A 100



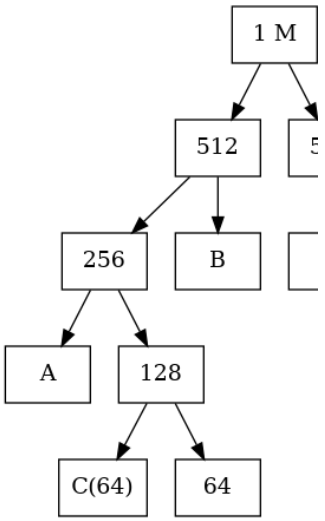
B 240



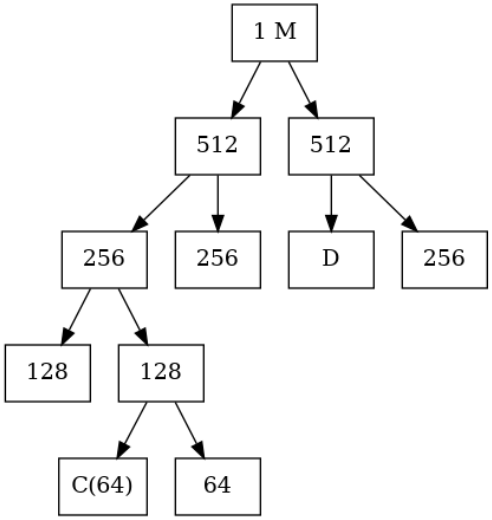
C 64



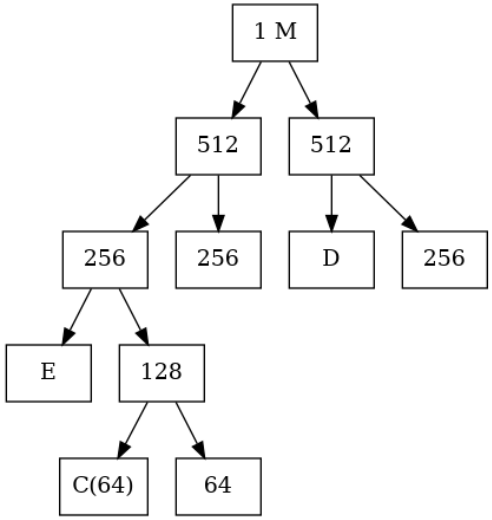
D 256



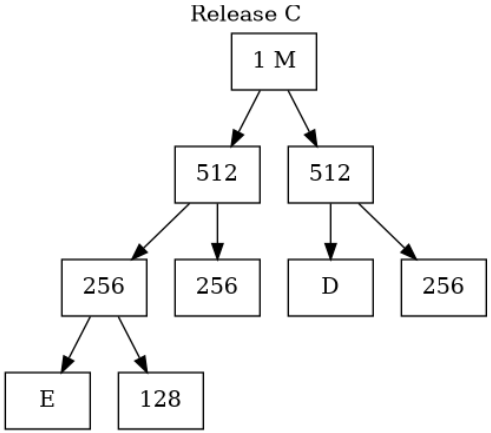
Release A and B



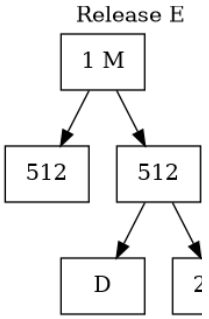
E 74



Release C



Release E



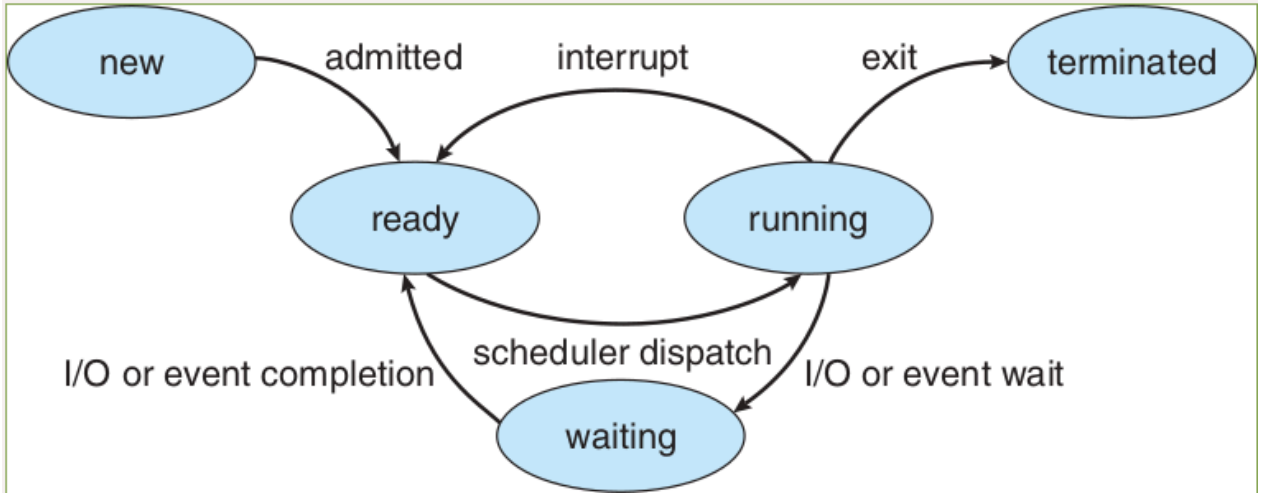
Buddy System Memory Management(III)

1. **A hybrid memory allocator** balances fixed and dynamic partitioning
    - dividing memory into partitions of base-2 sizes (powers of 2).
2. **The Allocation Process (Splitting)**
    - Memory blocks are sized as 2^k (e.g., 4KB, 8KB, 16KB).
    - If a process requests a size that is not a power of 2, the OS rounds up to the next highest power.
    - If only a larger block (e.g., 64KB) is available, the OS recursively splits it in half until the target size is reached:
        - 64KB splits into two 32KB buddies.
        - One 32KB splits into two 16KB buddies (one is allocated, one remains free).
3. **The Deallocation Process (Coalescing)**
    - When a block is freed, the OS checks the status of its specific "buddy".
    - If the buddy is also free, they immediately merge back into their parent size.
    - This process cascades upward automatically to form the largest possible free blocks.
4. **The Mathematical Advantage (Speed)**
    - The system is incredibly fast because finding a buddy requires no list searching.
    - The buddy of a block of size S at address A is located at exactly $A \oplus S$ (Bitwise XOR).
    - The OS calculates this directly in hardware.
5. **Pros:**
    - Extremely fast allocation
    - Coalescing
    - highly predictable performance.
6. **Cons:**
    - **Internal Fragmentation**
    - **External Fragmentation**
7. Usage
    1. The Linux Kernel
    2. Early UNIX Systems
    3. Modern Memory Allocators (jemalloc)
        - Used by FreeBSD and Facebook

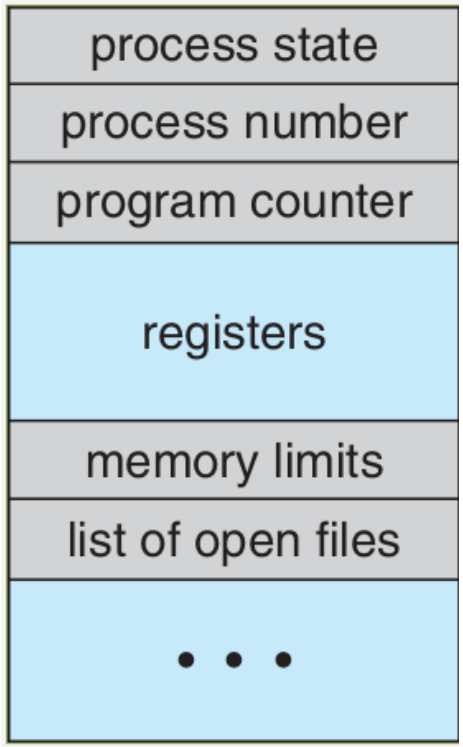
Process

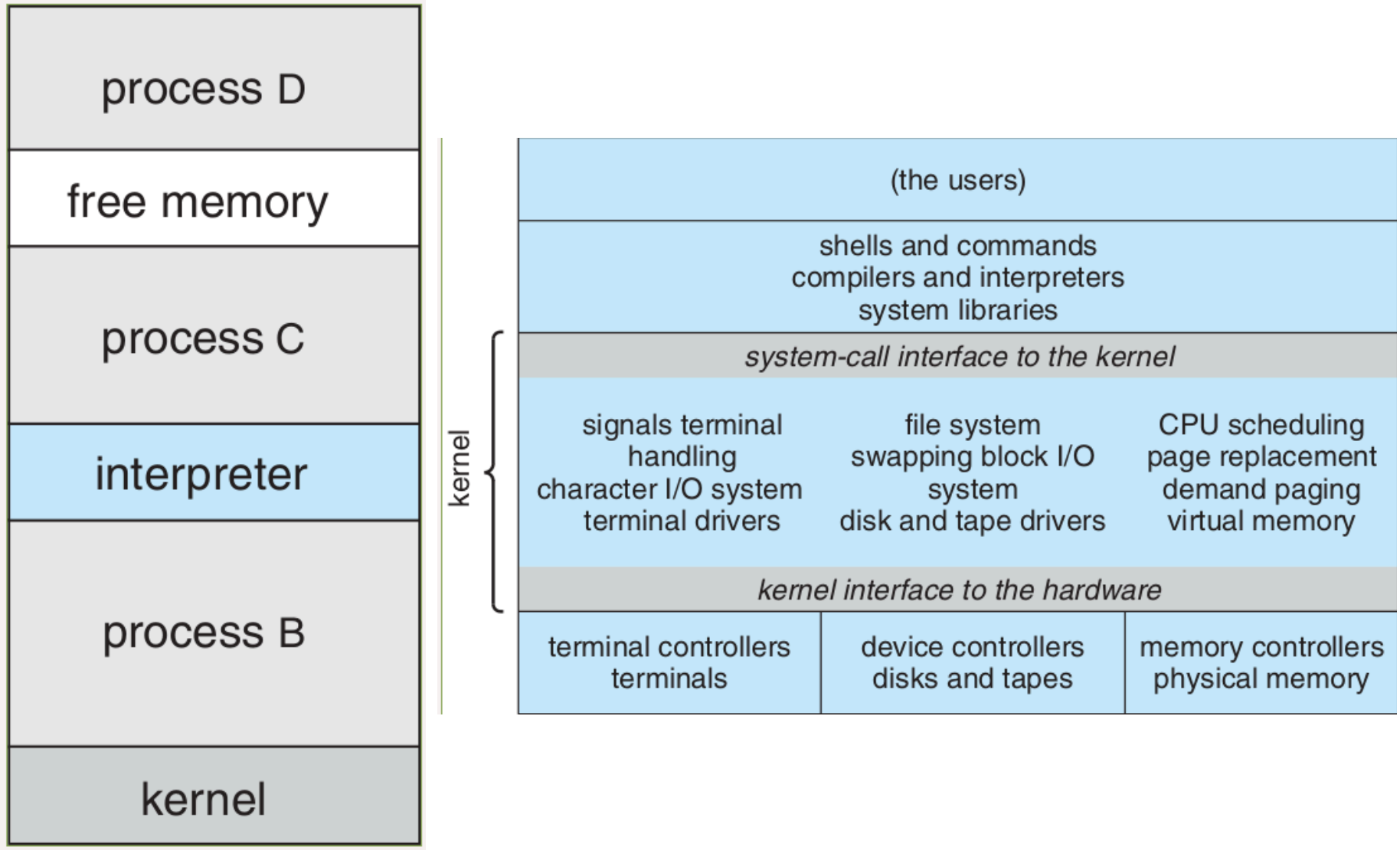
- Program
  - Place: Cards in card reader, file in disk, flash, etc.
  - passive
- Process
  - Place: Main Memory (RAM)
  - Active

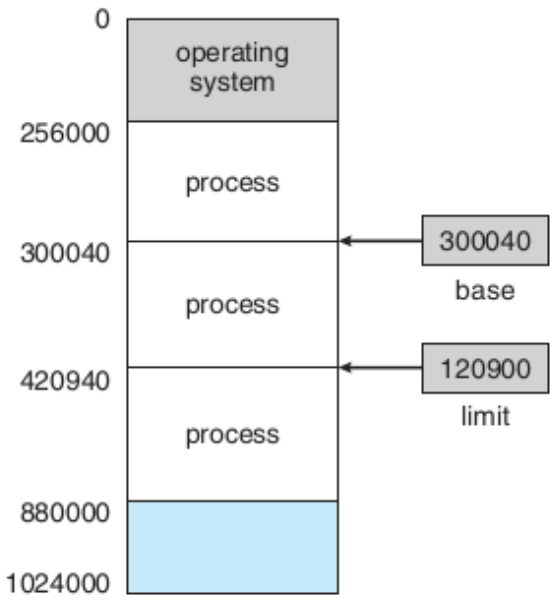
Process Status

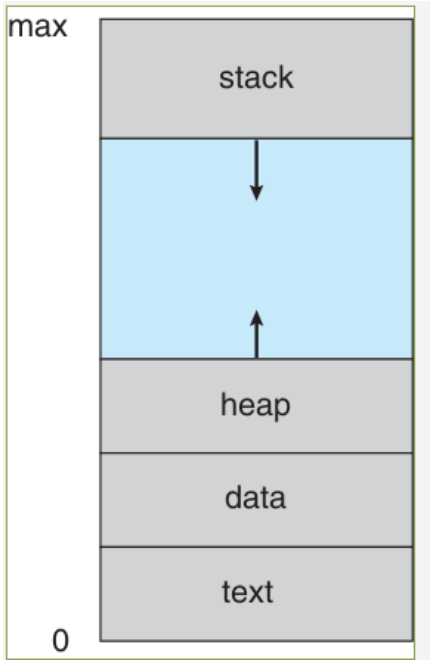


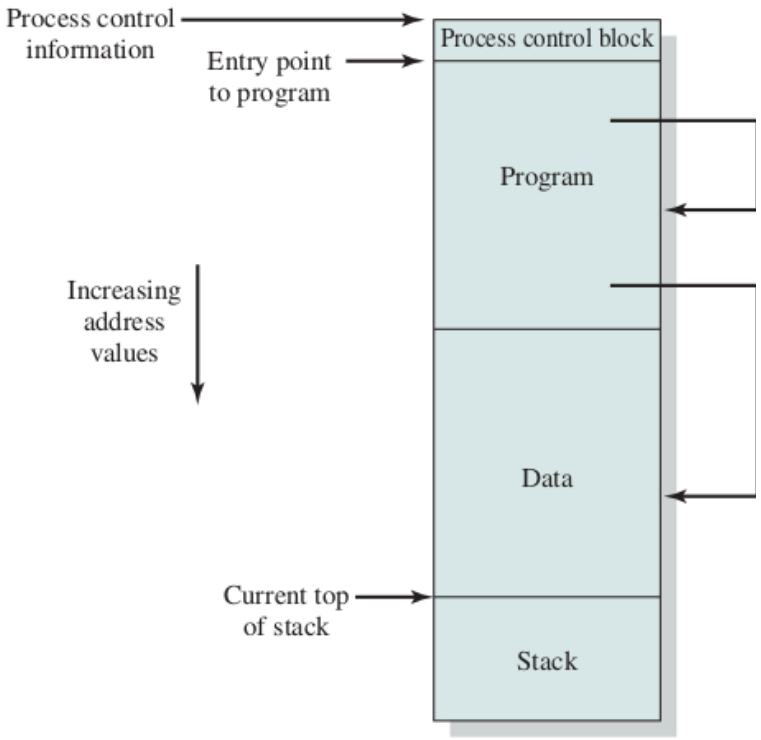
Process Control Block (PCB)

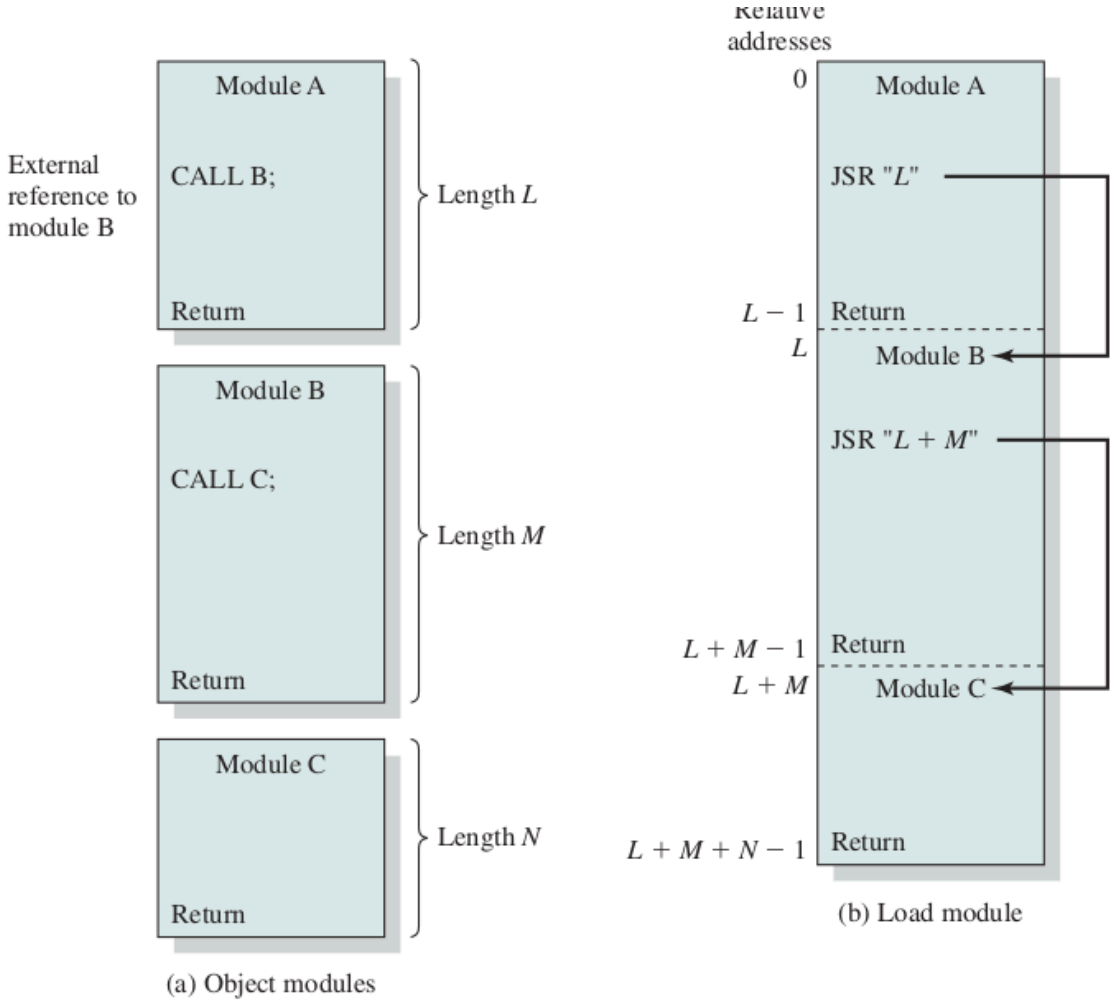



process D
free memory
process C
interpreter
process B
kernel
kernel
(the users)
shells and commands
compilers and interpreters
system libraries
system-call interface to the kernel
signals terminal handling
character I/O system
terminal drivers
file system
swapping block I/O system
disk and tape drivers
CPU scheduling
page replacement
demand paging
virtual memory
kernel interface to the hardware
terminal controllers
terminals
device controllers
disks and tapes
memory controllers
physical memory


0
operating system
256000
process
300040
process
420940
process
880000
1024000
300040
base
120900
limit

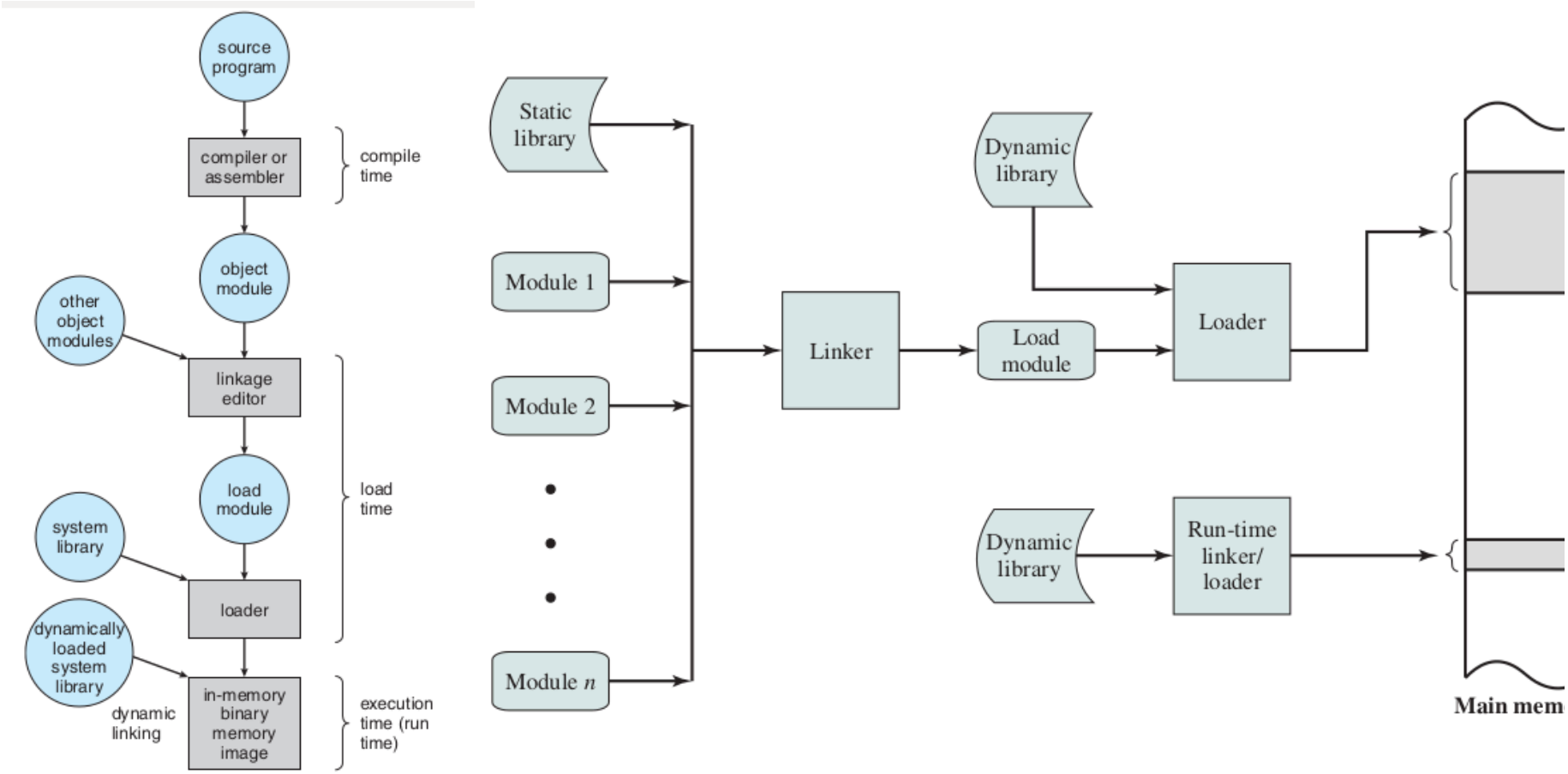

max
stack
heap
data
text
0


Process control information
Entry point to program
Process control block
Program
Increasing address values
Data
Current top of stack
Stack



(a) Object modules

(b) Load module


source program
compiler or assembler
compile time
object module
other object modules
linkage editor
load module
load time
system library
loader
dynamically loaded system library
dynamic linking
in-memory binary memory image
execution time (run time)
Static library
Module 1
Module 2
Module n
Linker
Dynamic library
Load module
Loader
Dynamic library
Run-time linker/ loader
Main mem

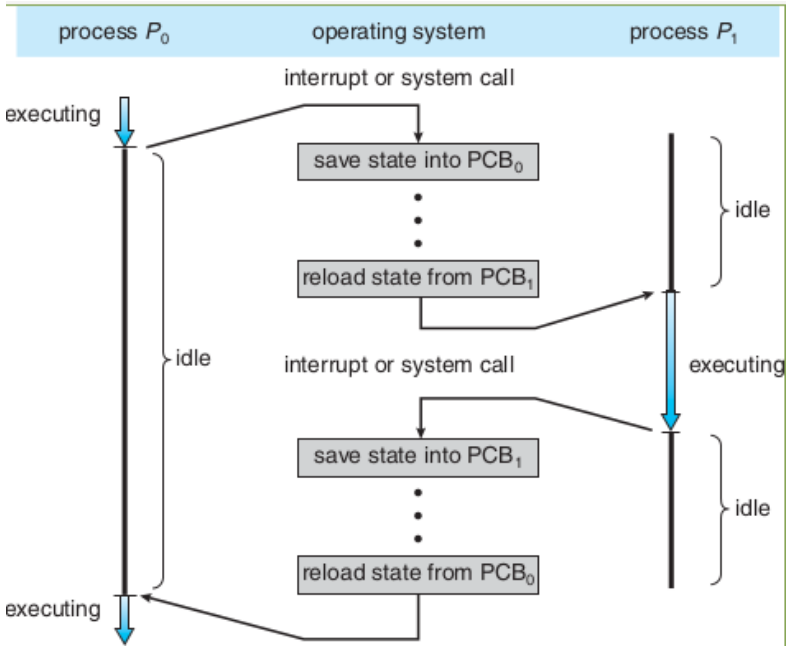
address binding, loader

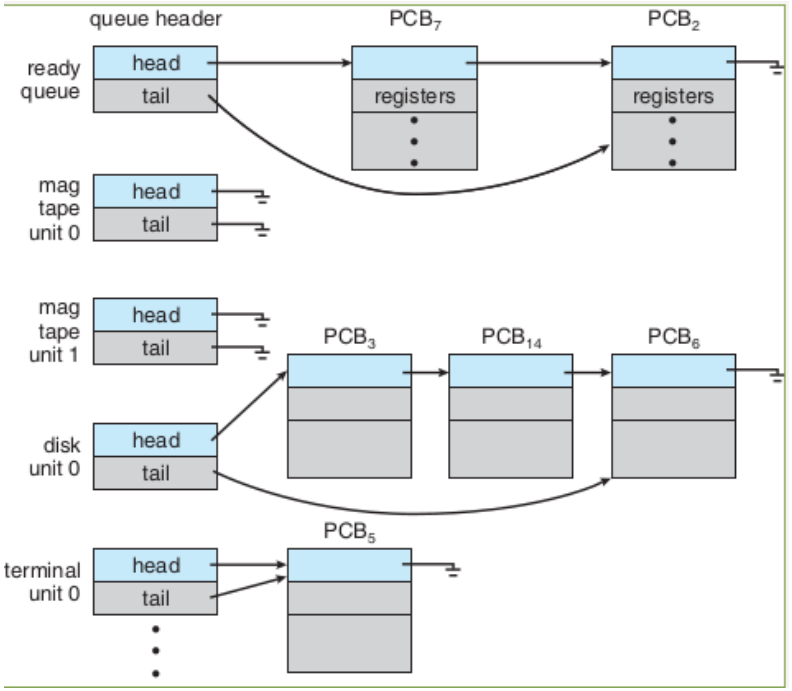
Binding Time	Function
Programming time	All actual physical addresses are directly specified by the programmer in the program itself.
Compile or assembly time	The program contains symbolic address references, and these are converted to actual physical addresses by the compiler or assembler.
Load time	The compiler or assembler produces relative addresses. The loader translates these to absolute addresses at the time of program loading.
Run time	The loaded program retains relative addresses. These are converted dynamically to absolute addresses by processor hardware.

address binding, linker

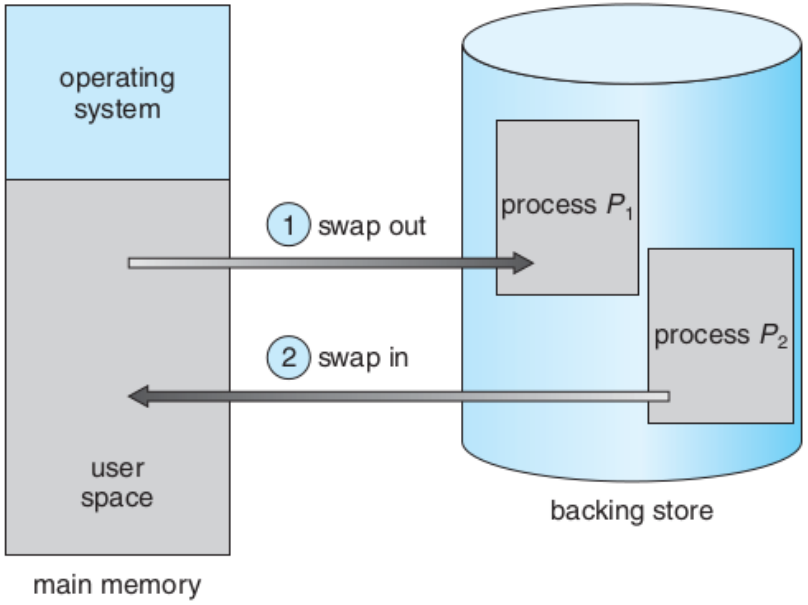
Linkage Time	Function
Programming time	No external program or data references are allowed. The programmer must place into the program the source code for all subprograms that are referenced.
Compile or assembly time	The assembler must fetch the source code of every subroutine that is referenced and assemble them as a unit.
Load module creation	All object modules have been assembled using relative addresses. These modules are linked together and all references are restated relative to the origin of the final load module.
Load time	External references are not resolved until the load module is to be loaded into main memory. At that time, referenced dynamic link modules are appended to the load module, and the entire package is loaded into main or virtual memory.
Run time	External references are not resolved until the external call is executed by the processor. At that time, the process is interrupted and the desired module is linked to the calling program.

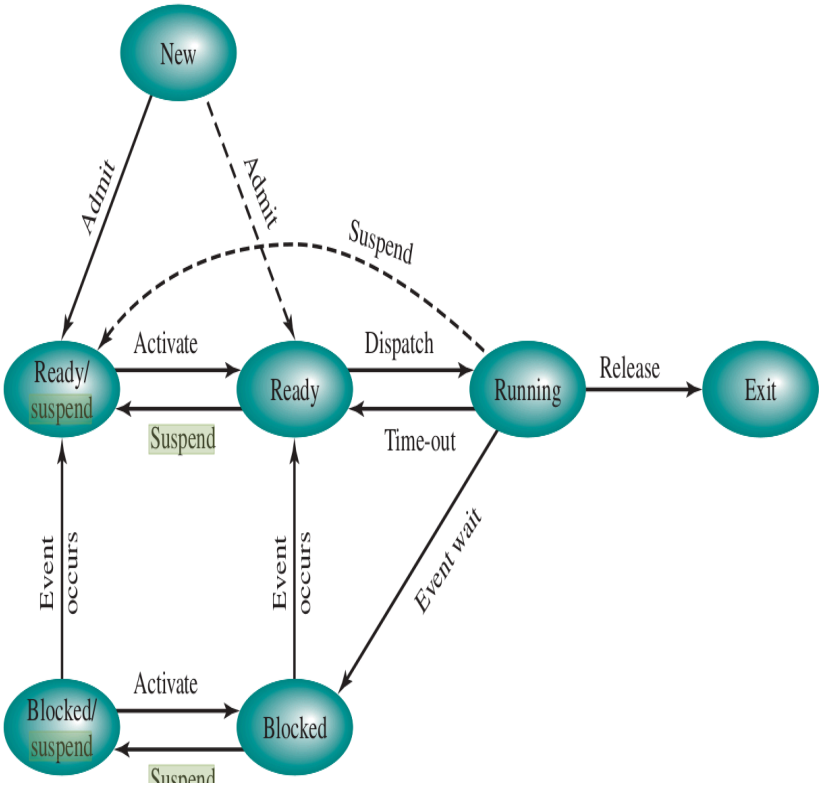
Blocked or Waiting





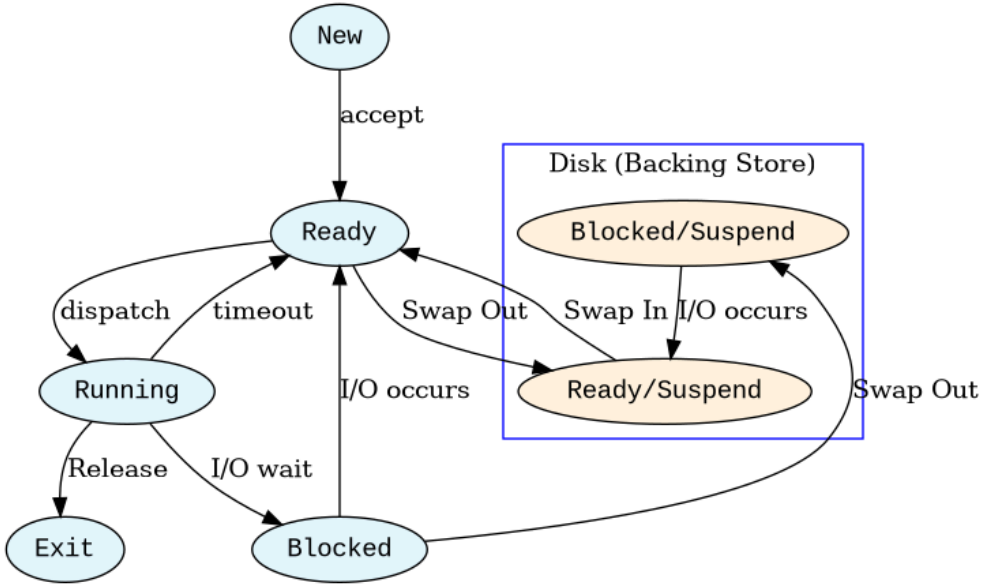
Process Suspension



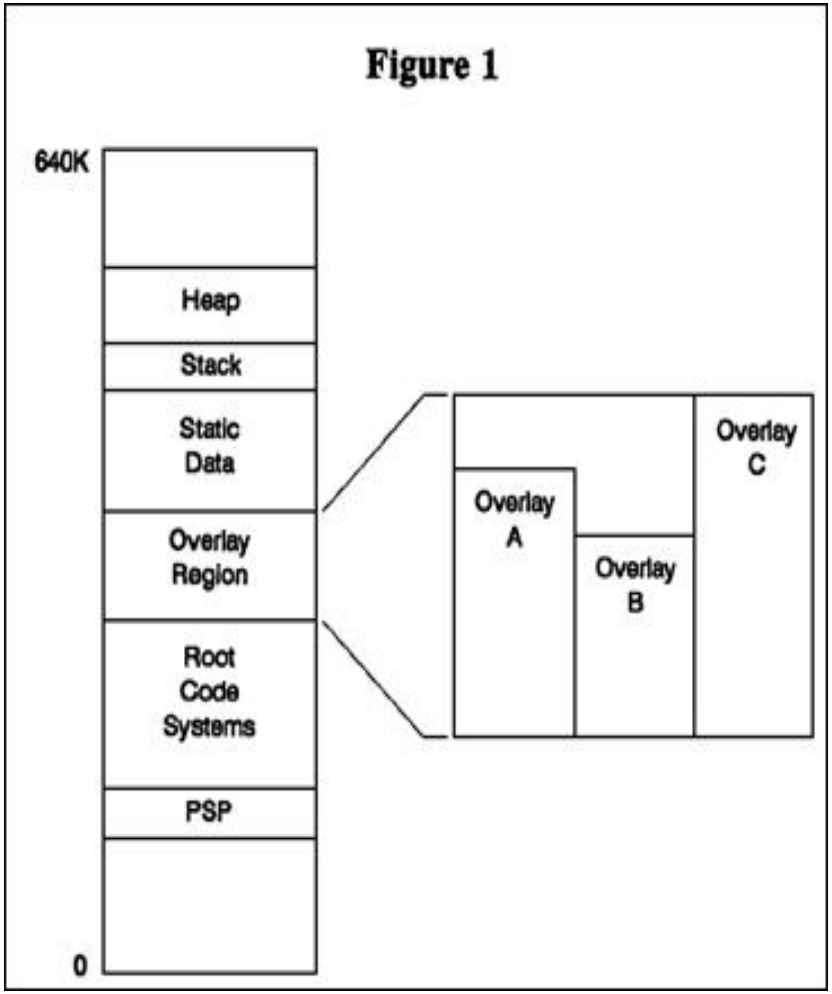


Process Suspension (Swapping)

1. **The 7-State Model Transitions**
    - **Blocked → Blocked/Suspend**
    - **Ready → Ready/Suspend**
    - **Blocked/Suspend → Ready/Suspend**
2. **Reasons for Suspension**
    - **Swapping**
    - **User Request**
    - **Parent Request**
    - **Timing**
3. **Pros**
    - Increases the degree of multiprogramming.
    - Frees space for higher-priority or "Ready" processes.
4. **Cons**
    - **High Latency**
    - **Thrashing**

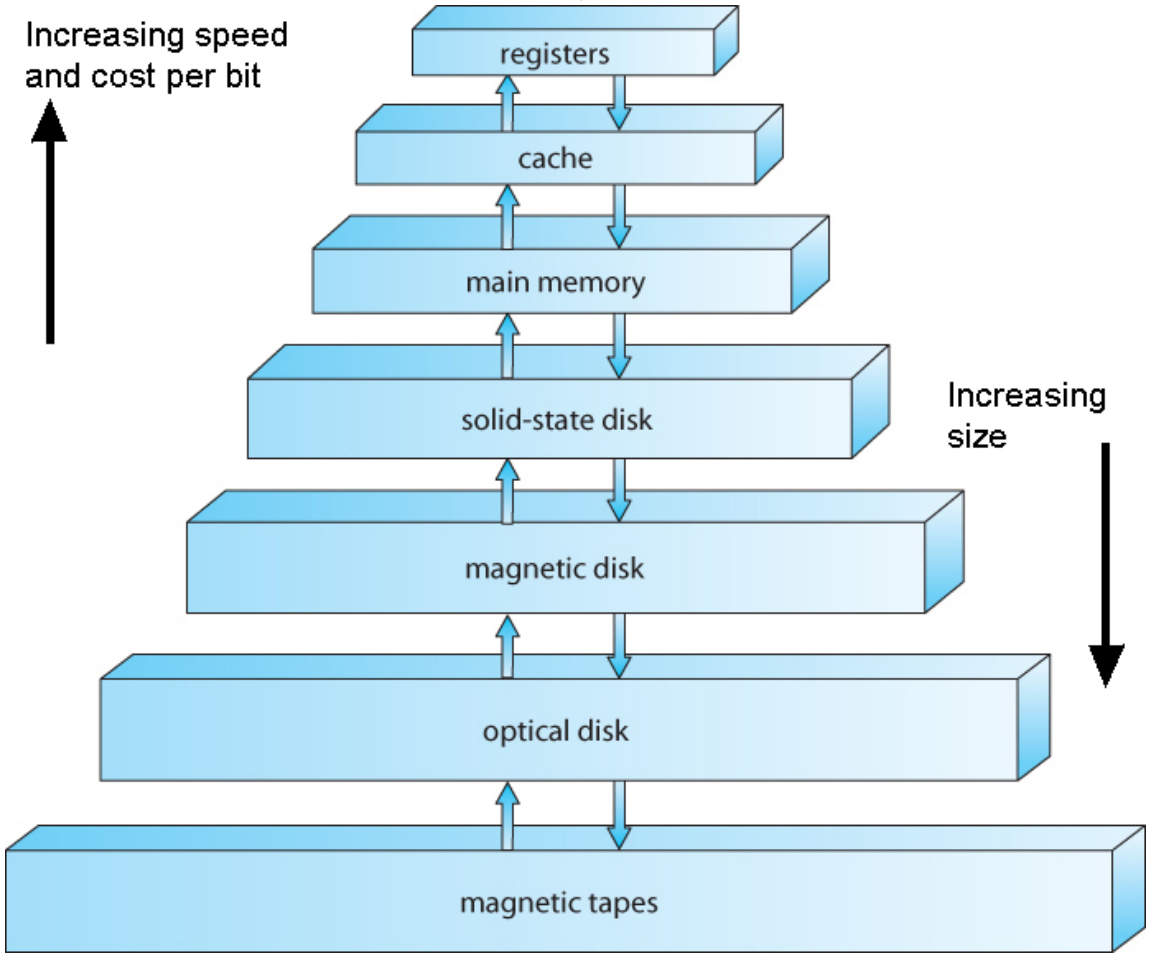


Memory Overlays

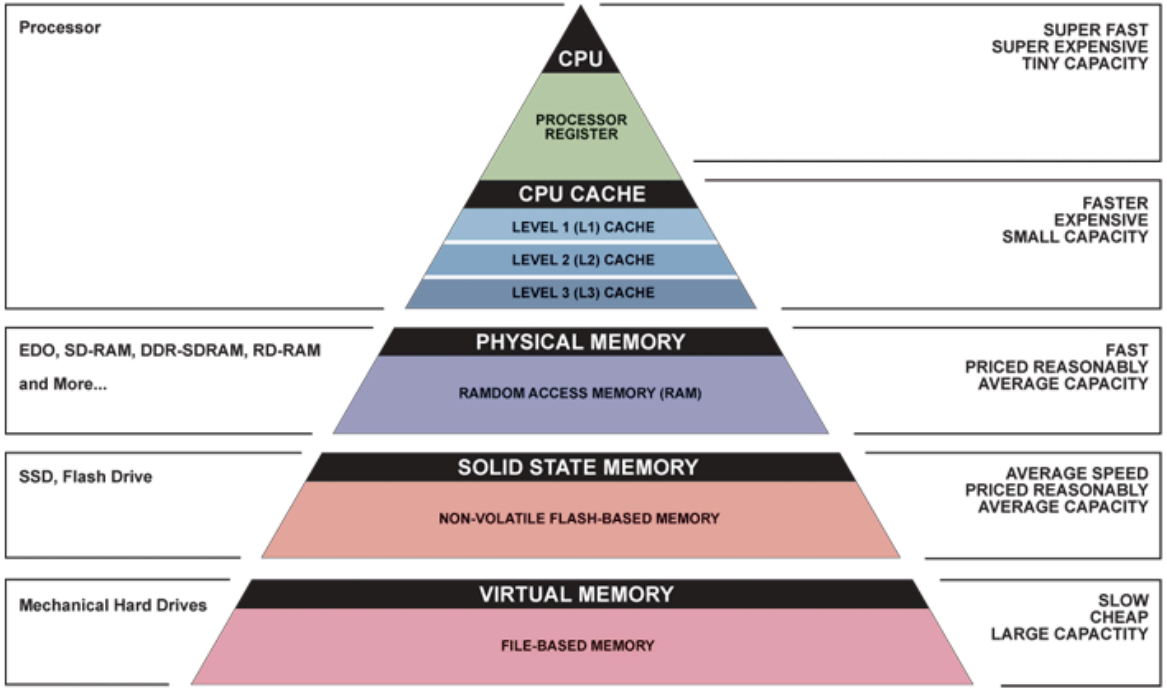


1. **Architectural Components**
2. **The Execution Process**
3. **Pros**:
    - **zero hardware or MMU support**
    - Work on small memory
4. **Cons**:
    - **Massive Programmer Burden**
    - Difficult to debug
    - Hard to modularize
    - Hard to upgrade.
5. The Mainframe Era (1960s – 1970s)
    - 16KB to 64KB
    - IBM was the undisputed king
    - 1964, 16KB, Transients
    - UNIVAC & Sperry Rand
    - NASA & The Aerospace
    - Virtual Memory by Paging
6. The Microcomputer/PC Era, 1980s, early 1990s
    - Intel 8086/8088
    - MS-DOS / PC DOS/ Dr Dos
    - Borland, Turbo Pascal/Turbo C
    - Lotus 1-2-3
7. The Extinction of Overlays
    - Virtual Memory by Paging
    - Intel Protected Mode
    - Windows 95 and Linux
    - OS tracking memory entirely

سلسله مراتب حافظه

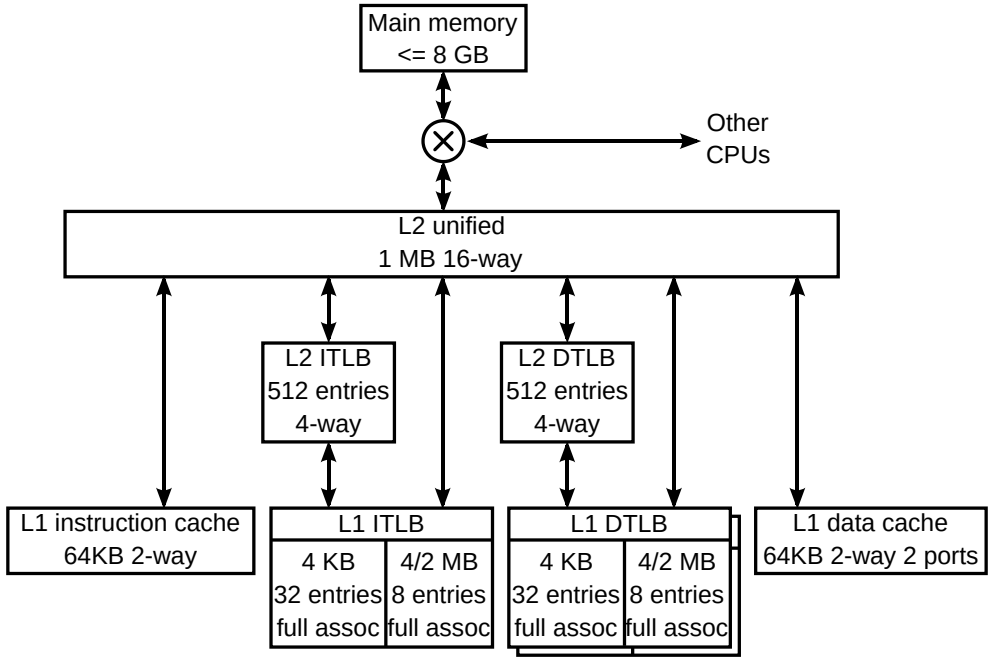


سلسله مراتب حافظه جزئی‌تر

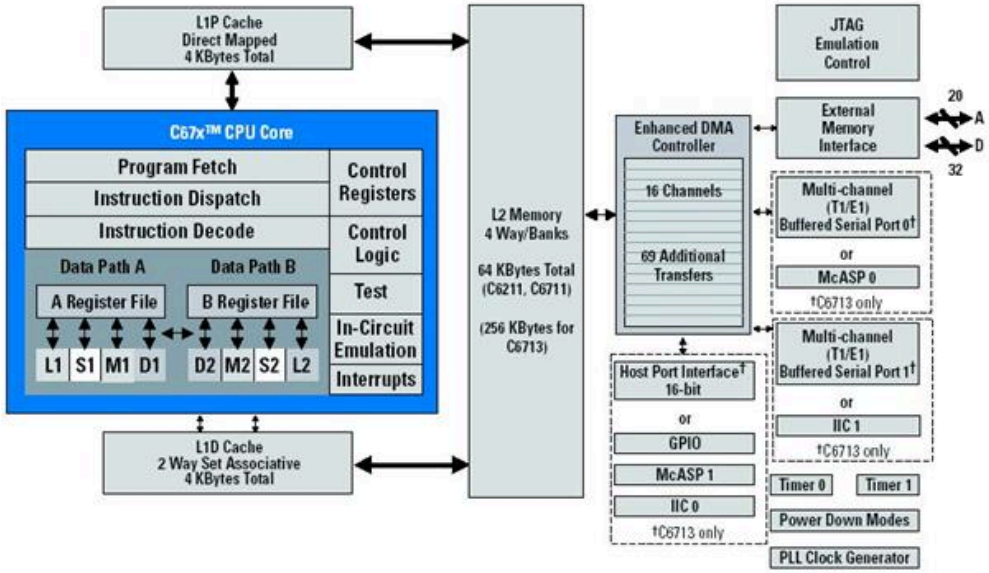


▲ Simplified Computer Memory Hierarchy
Illustration: Ryan J. Leng

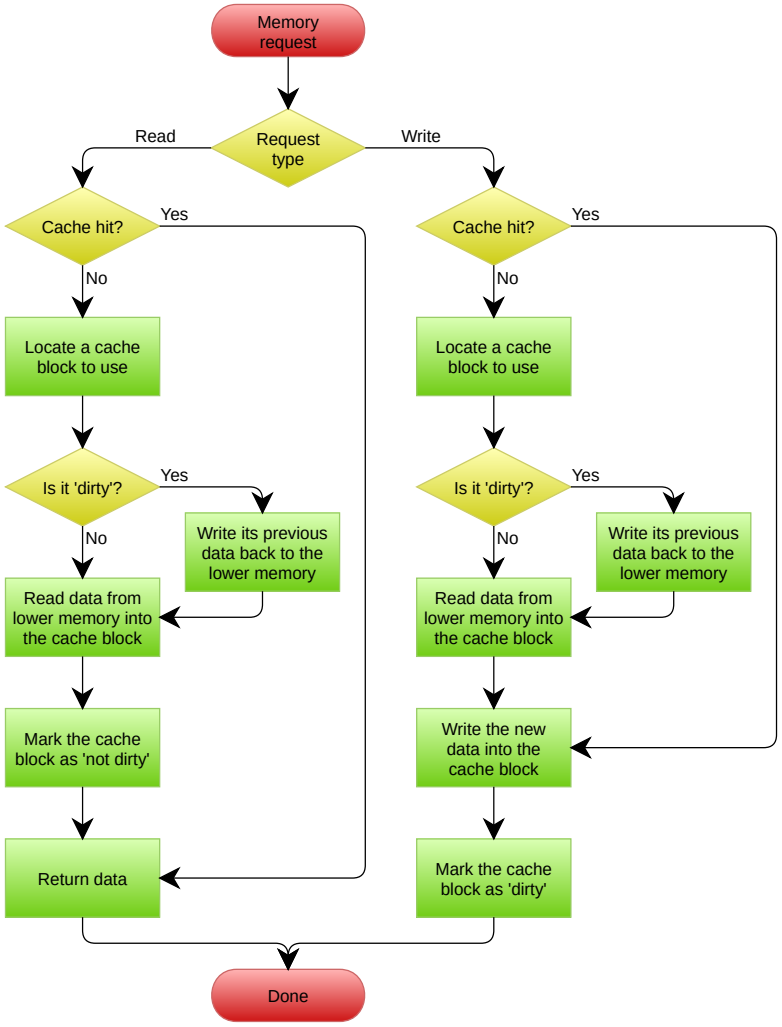
حافظة نهان


Main memory
<= 8 GB
Other
CPUs
L2 unified
1 MB 16-way
L2 ITLB
512 entries
4-way
L2 DTLB
512 entries
4-way
L1 instruction cache
64KB 2-way
L1 ITLB
4 KB
32 entries
full assoc
4/2 MB
8 entries
full assoc
L1 DTLB
4 KB
32 entries
full assoc
4/2 MB
8 entries
full assoc
L1 data cache
64KB 2-way 2 ports

حافظهٔ نهان دو سطحی در یک پردازندهٔ واقعی



الگوریتم خواندن و نوشتن از حافظهٔ نهان



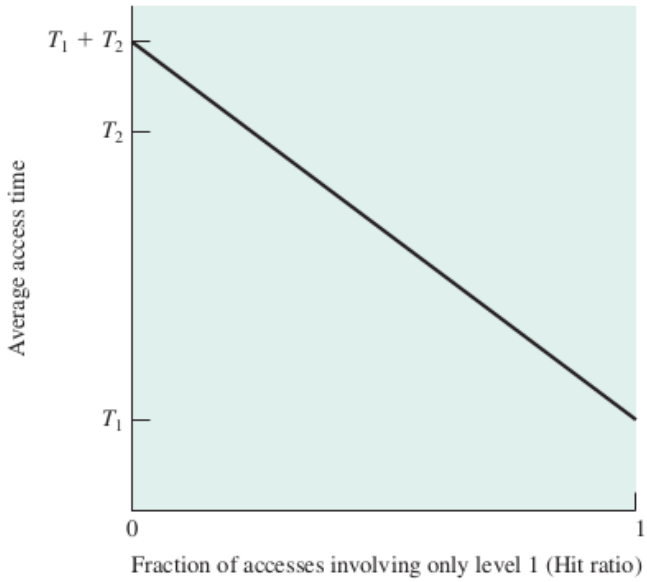
Effective Access Time (EAT)

- t_m : زمان دسترسی به حافظه‌ی اصلی
- t_c : زمان دسترسی به حافظه‌ی نهان
- h_c : ضریب اصابت به حافظه‌ی نهان

$$EAT = h_c * t_c + (1 - h_c) * (t_m + t_c)$$

اگر ضریب اصابت (یا نسبت اصابت) برای پردازنده‌ای 0.95 باشد و سرعت دسترسی به حافظهٔ اصلی 100 میکرو ثانیه باشد و سرعت دسترسی حافظهٔ نهان 1 میکرو ثانیه باشد در این صورت زمان دسترسی مؤثر برابر خواهد بود با

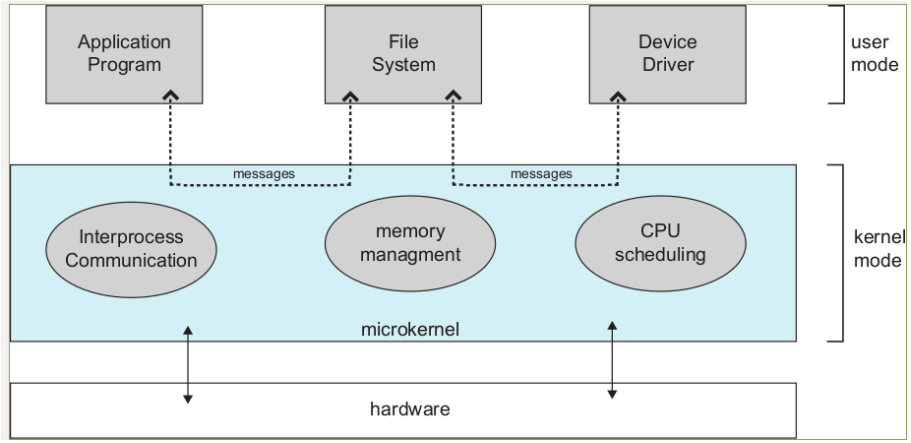
- EAT = 0.95 * 1 + (1 − 0.95) * (100 + 1)
- EAT = 0.95 + 0.05 * 101
- EAT = 0.95 + 5.05
- EAT = 6 µs



ms	µs	ns	action
		0.5	CPU L1 dCACHE reference
		1	speed-of-light (a photon) travel a 1 ft (30.5cm) distance
		5	CPU L1 iCACHE Branch mispredict
		7	CPU L2 CACHE reference
		71	CPU cross-QPI/NUMA best case on XEON E5-46
		100	MUTEX lock/unlock
		100	own DDR MEMORY reference
	20	000	Send 2K bytes over 1 Gbps NETWORK
	250	000	Read 1 MB sequentially from MEMORY
10	000	000	DISK seek
10	000	000	Read 1 MB sequentially from NETWORK
30	000	000	Read 1 MB sequentially from DISK
150	000	000	Send a NETWORK packet CA -> Netherlands

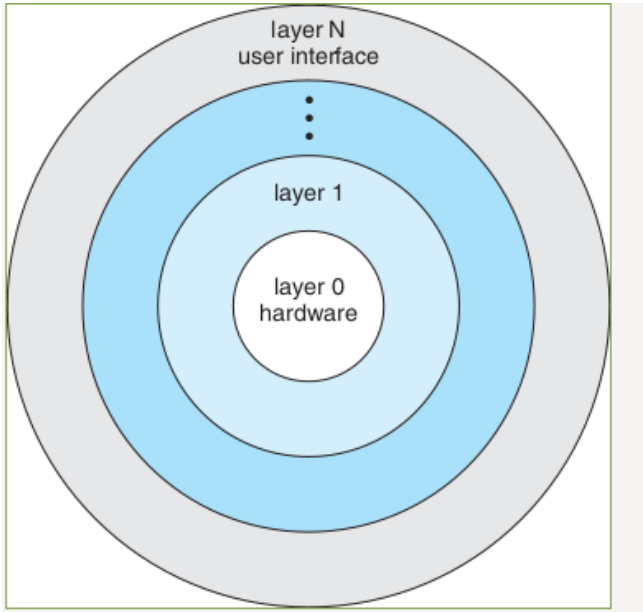
[link]

Microkernel Architecture



1. Core Responsibilities (Inside Kernel Space)
    - **Low-Level Memory Management**
    - **Thread Scheduling**
    - **Inter-Process Communication (IPC)**
2. **Architecture Mechanics**
    - Services run as isolated user processes
    - Application cannot make a direct system call
    - Send an IPC message *through* the microkernel
    - Forwards it to the File Server
3. **Pros**:
    - **High Reliability & Isolation**
    - **Security**
    - **Portability & Extensibility**
4. **Cons**:
    - **Performance Overhead**
5. History
    - The Mach kernel
    - core macOS/iOS XNU hybrid
    - QNX
    - seL4
    - The Tanenbaum-Torvalds Debate
        - MINIX
    - safety and stability.

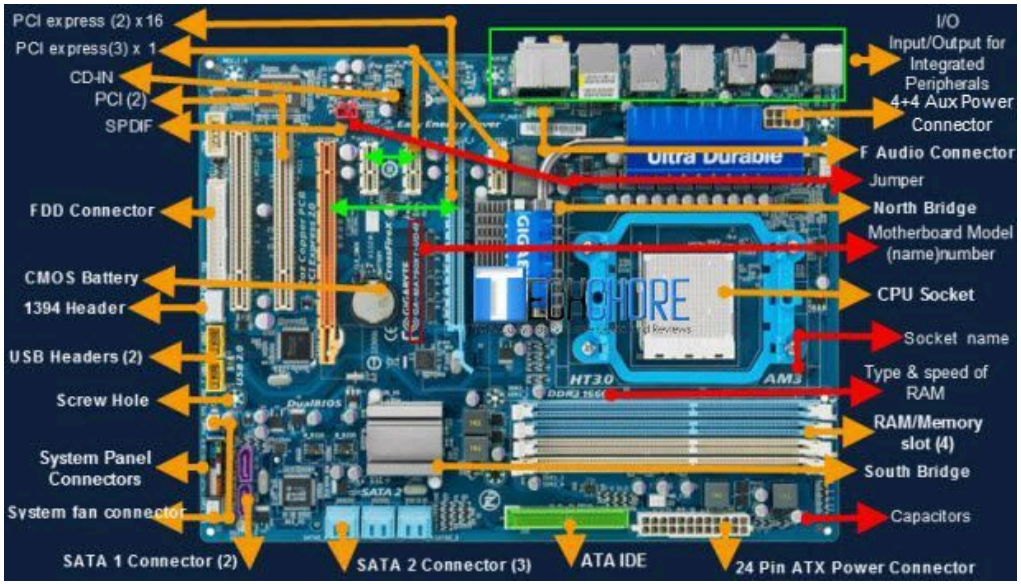
Multi Layer



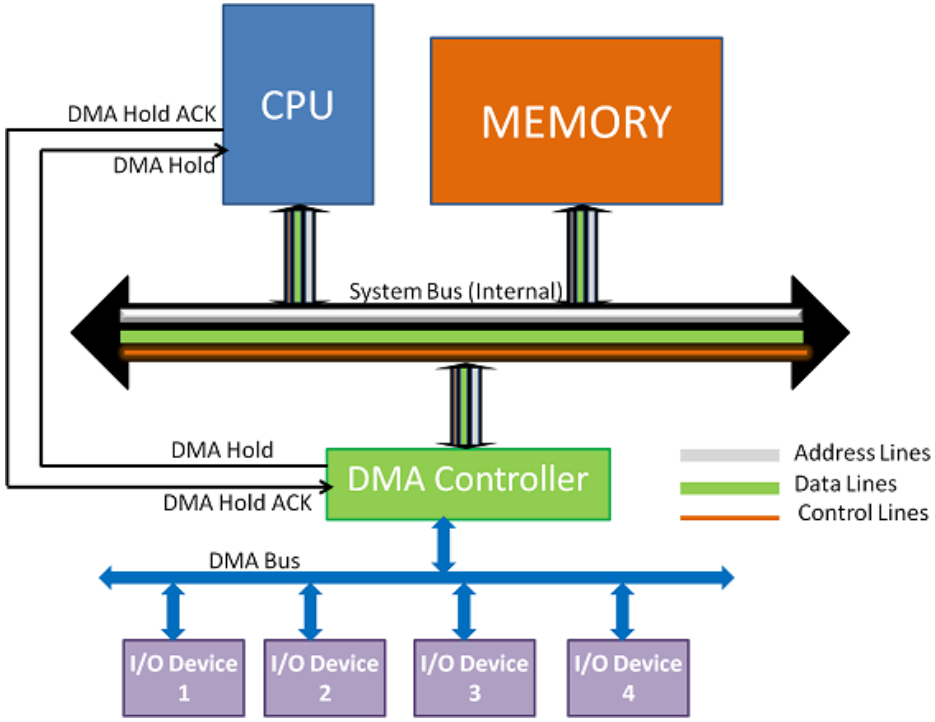
Effects on current situations

Monolithic

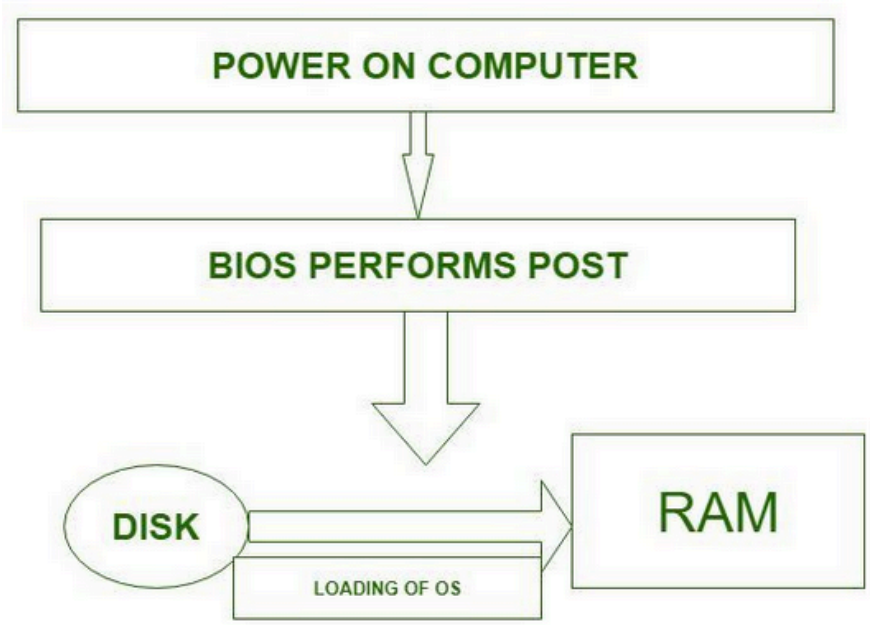
Motherboard

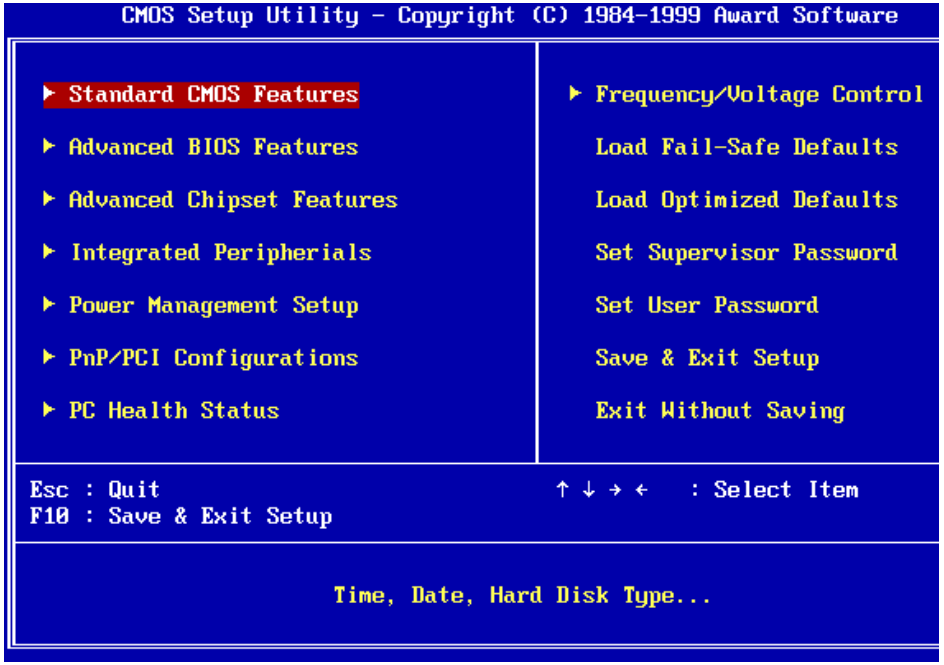


Direct Memory Access(DMA)

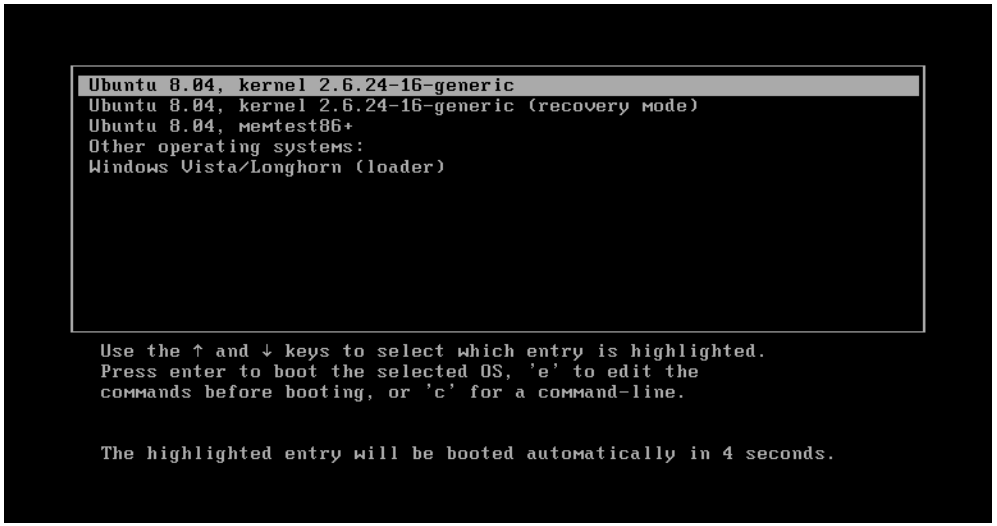


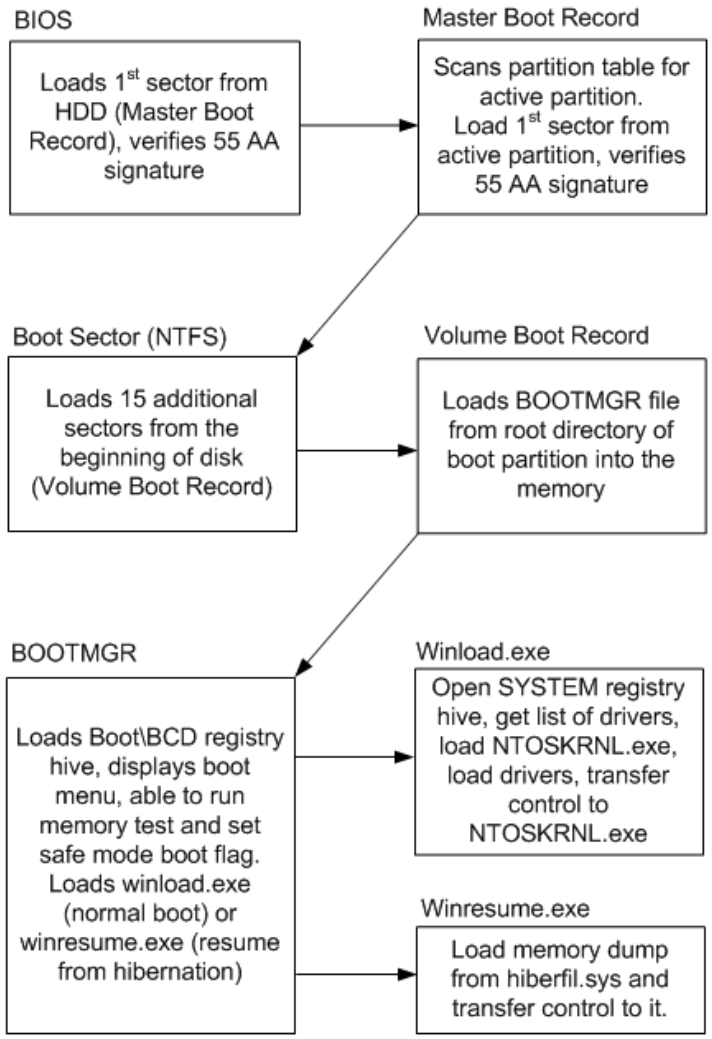
BIOS


POWER ON COMPUTER
BIOS PERFORMS POST
DISK
RAM
LOADING OF OS


CMOS Setup Utility - Copyright (C) 1984-1999 Award Software
Standard CMOS Features
Advanced BIOS Features
Advanced Chipset Features
Integrated Peripherials
Power Management Setup
PnP/PCI Configurations
PC Health Status
Frequency/Voltage Control
Load Fail-Safe Defaults
Load Optimized Defaults
Set Supervisor Password
Set User Password
Save & Exit Setup
Exit Without Saving
Esc : Quit
F10 : Save & Exit Setup
↑ ↓ → ← : Select Item
Time, Date, Hard Disk Type...

Boot sequence





END

References(I)

- https://stackoverflow.com/questions/18550370/calculate-the-effective-access-time
- http://os-book.com/
- https://en.wikipedia.org/wiki/Paging
- Sean K. Barker (https://tildesites.bowdoin.edu/~sbarker/)
    - https://tildesites.bowdoin.edu/~sbarker/teaching/courses/os/14spring/slides/lec12.pdf
    - https://tildesites.bowdoin.edu/~sbarker/teaching/courses/os/14spring/lectures.html
- https://en.wikipedia.org/wiki/Page_(computer_memory)
- http://blog.cs.miami.edu/burt/2012/10/31/virtual-memory-pages-and-page-frames/
- https://www.tldp.org/LDP/tlk/mm/memory.html
- https://www.cse.iitb.ac.in/~mythili/teaching/cs347_autumn2016/notes/07-memory.pdf

References(II)

- https://www.kernel.org/doc/html/latest/admin-guide/mm/index.html
- https://www.geeksforgeeks.org/operating-system-paging/
- https://samypesse.gitbooks.io/how-to-create-an-operating-system/Chapter-8/
- https://www.javatpoint.com/os-segmented-paging
- https://www.geeksforgeeks.org/difference-between-internal-and-external-fragmentation/
- https://web.fe.up.pt/~arestivo/presentation/os-memory/#15
- https://binaryterms.com/contiguous-memory-allocation-in-operating-system.html
- https://github.com/mor1/ia-operating-systems/wiki/06-Virtual-Addressing

References(III)

- https://github.com/mor1/ia-operating-systems
- https://www.faceprep.in/operating-systems/operating-systems-fragmentation-and-compaction/
- https://slideplayer.com/slide/7084682/
- https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/images/Chapter1/1_4_StorageDeviceHierarchy.jpg
- http://images.bit-tech.net/content_images/2007/11/the_secrets_of_pc_memory_part_1/hei.png
- https://en.wikipedia.org/wiki/Cache_(computing)
- https://www.byclb.com/TR/Tutorials/dsp_advanced/ch1_1_dosyalar/image025.jpg
- https://en.wikipedia.org/wiki/File:Cache,hierarchy-example.svg
- https://en.wikipedia.org/wiki/CPU_cache
- https://tutorialspoint.dev/image/Translation.png

References(IV)

- https://www.cs.princeton.edu/courses/archive/spr11/cos217/lectures/18MemoryMgmt.pdf
- http://harmanani.github.io/classes/csc320/Notes/ch05.pdf
- https://www.cs.princeton.edu/courses/archive/spr11/cos217/lectures/18MemoryMgmt.pdf
- http://harmanani.github.io/classes/csc320/Notes/ch05.pdf
- https://www.gatevidyalay.com/translation-lookaside-buffer-tlb-paging/
- https://www.amazon.com/ASUS-DDR3-Intel-Motherboard-H61M/dp/B00BN36V4W
- https://www.asus.com/Motherboards-Components/Motherboards/Workstation/P10S-WS/
- https://commons.wikimedia.org/wiki/File:Intel_D945GCCR_Socket_775.png

References(V)

- https://witscad.com/course/computer-architecture/chapter/dma-controller-and-io-processor
- https://www.uou.ac.in/lecturenotes/computer-science/BCA-17/Computer%20Organization%20Part%202.pdf
- https://www.pvpsiddhartha.ac.in/dep_it/lecturenotes/CSA/unit-5.pdf
- https://toshiba.semicon-storage.com/us/semiconductor/knowledge/e-learning/micro-intro/chapter4/interrupt-processing-types-interrupts.html
- https://stackoverflow.com/questions/4087280/approximate-cost-to-access-various-caches-and-main-memory#4087315
- https://codex.cs.yale.edu/avi/os-book/